

Dijkstra's Algorithm: Shortest Paths, Fully Proven

Programming and Data Structures — Weeks 12-13, Lecture 1

Dr. Innocent Nyalala

Objectives

By the end of this lecture, you will be able to implement Dijkstra's algorithm in full, using Week 8's priority queue directly; trace it completely on a concrete weighted graph, including reconstructing the actual shortest path, not merely its length; state and prove the correctness argument that resolves Week 11 Lecture 4's demonstrated BFS failure on weighted graphs; and state the algorithm's exact cost, explaining precisely why the choice of priority queue implementation affects it.

Outline

Today covers a postal-delivery-route analogy motivating weighted shortest paths; Dijkstra's algorithm in full, built directly on Week 8's priority queue; a complete trace including path reconstruction; the correctness proof explaining why greedy finalization is valid here; a direct resolution of Week 11 Lecture 4's demonstrated BFS failure; a precise cost analysis and why priority queue implementation choice matters; the MTech-track connection to Week 10's greedy algorithms; an in-class quiz; and a summary.

The postal-delivery-route analogy

A postal service planning delivery routes needs the actual fastest route between its sorting depot and every delivery zone, measured in real travel time, not the route with the fewest turns or intersections. Week 11 Lecture 4 proved concretely that BFS answers the wrong question for this need, optimizing hop count rather than total weight. Dijkstra's algorithm, covered in full this lecture, answers the genuinely correct question: the minimum total travel time from one starting point to every other reachable location in the network.

Dijkstra's algorithm, in full

```
import heapq

def dijkstra(graph, start, n):
    dist = {start: 0}
    prev = {start: None}
    visited = set()
    pq = [(0, start)]  # Week 8's priority queue, via heapq
    while pq:
        d, node = heapq.heappop(pq)
        if node in visited:
            continue
        visited.add(node)
        for neighbor, weight in graph.adj[node]:
            if neighbor in visited:
                continue
            new_dist = d + weight
            if new_dist < dist.get(neighbor, float('inf')):
                dist[neighbor] = new_dist
                prev[neighbor] = node
                heapq.heappush(pq, (new_dist, neighbor))
    return dist, prev
```

dist tracks the best known distance to each node discovered so far, only ever improving as strictly better paths are found. prev tracks which node immediately precedes each node on its current best-known path — this is exactly Week 9 Lecture 2's reconstruction technique, applied here to record enough information to recover the actual shortest path afterward, not merely its length. The priority queue, directly reusing Week 8's structure, always yields whichever unvisited node currently has the smallest known distance — this single replacement, priority queue instead of BFS's plain FIFO queue, is the entire mechanism that fixes Week 11's demonstrated failure.

Tracing Dijkstra's completely on Week 11's counterexample graph

Graph: edge(0,1,10), edge(0,2,1), edge(2,1,1) -- Week 11 Lecture 4's exact example

dist={0:0}, pq=[(0,0)]

Pop (0,0). visited={0}. Neighbors: 1 (d=10, better than inf → dist[1]=10, push (10,1)); 2 (d=1, better than inf → dist[2]=1, push (1,2))

Pop (1,2). visited={0,2}. Neighbor 1: d=1+1=2, BETTER than current dist[1]=10 → dist[1]=2, push (2,1)

Pop (2,1). visited={0,2,1}. No unvisited neighbors.

Pop (10,1). node 1 ALREADY visited → skip (stale queue entry)

Final: dist={0:0, 1:2, 2:1}. prev={0:None, 1:2, 2:0}

Tracing Dijkstra's algorithm completely on Week 11 Lecture 4's exact counterexample graph: starting from node 0, both neighbors are discovered, with node 1 initially recorded at distance 10 via the direct edge. Popping node 2 next (distance 1, smaller than node 1's current 10) discovers a better path to node 1, through node 2, at distance $1 + 1 = 2$ — strictly improving the earlier estimate. This directly resolves Week 11's demonstrated failure: Dijkstra's correctly reports $\text{dist}[1] = 2$, the true shortest distance, not BFS's incorrect hop-count-based answer of effectively preferring the direct, more expensive edge.

Reconstructing the actual path, not just its length

```
def reconstruct_path(prev, target):
    path = []
    node = target
    while node is not None:
        path.append(node)
        node = prev[node]
    return path[::-1]

path_to_1 = reconstruct_path(prev, 1)
print(path_to_1)    # [0, 2, 1] - the genuine shortest path, not the direct edge
```

Reconstructing the actual path walks the prev array backward from the target node to the start, then reverses the result — precisely Week 9 Lecture 2's reconstruction technique, reapplied here in a genuinely different setting. This confirms the algorithm produces not merely the correct shortest distance, 2, but also the actual route achieving it, $0 \rightarrow 2 \rightarrow 1$ — directly answering “what is the fastest route,” not merely “how long does the fastest route take.”

Tracing a slightly larger example, completely

Graph: edge(0,1,4), edge(0,2,1), edge(2,1,2), edge(1,3,1), edge(2,3,5)

```
dist={0:0}, pq=[(0,0)]
Pop(0,0). visited={0}. push (4,1) dist[1]=4; push (1,2) dist[2]=1
Pop(1,2). visited={0,2}. neighbor1: 1+2=3 < 4 → dist[1]=3, prev[1]=2, push(3,1)
      neighbor3: 1+5=6 → dist[3]=6, prev[3]=2, push(6,3)
```

```

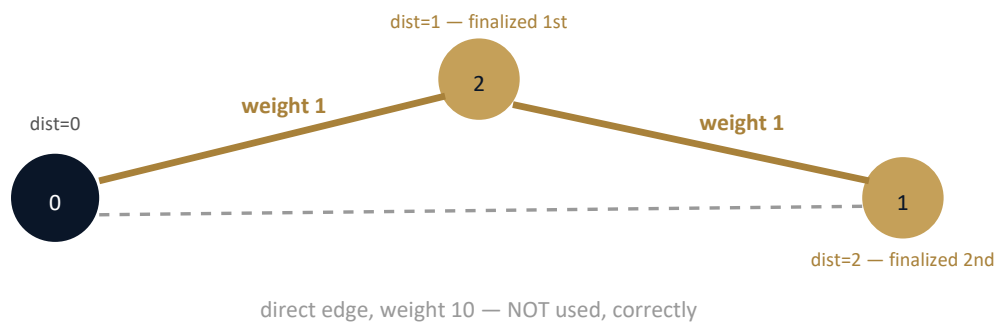
Pop(3,1). visited={0,2,1}. neighbor3: 3+1=4 < 6 → dist[3]=4, prev[3]=1, push(4,3)
Pop(4,1). node1 already visited → skip (stale)
Pop(4,3). visited={0,2,1,3}. no unvisited neighbors
Pop(6,3). node3 already visited → skip (stale)

```

Final: `dist={0:0,1:3,2:1,3:4}`

Tracing a slightly larger 4-node example completely: the initial direct edge to node 1 (weight 4) is later improved to 3, via node 2, exactly the same “discover a better path after an initial estimate” pattern from the smaller example. Node 3’s distance is similarly revised, from an initial 6 (via node 2 directly) down to the true optimal 4 (via node 1). This confirms the algorithm’s revision mechanism generalizes correctly to graphs with more than 3 nodes and more than one possible revision per node, not merely the minimal case traced first.

The picture: Dijkstra’s expanding frontier



Dijkstra's frontier expands by true accumulated weight, finalizing node 2 (dist 1) before node 1 (dist 2 via 2, not 10 direct).

Unlike BFS’s frontier, which expands strictly by hop count, Dijkstra’s frontier expands strictly by true accumulated distance — nodes are finalized in order of increasing shortest distance, which is precisely what guarantees correctness even when edge weights differ substantially.

The correctness proof: why greedy finalization works

The correctness claim: when a node is popped from the priority queue and found not yet visited, its current `dist` value is already its true shortest distance, permanently, never requiring later revision. The proof sketch: suppose some node u is popped with a `dist[u]` that is not actually optimal. Any genuinely shorter path to u must pass through some other, unvisited node v along the way, and since all edge weights are non-negative, $dist[v]$ must be strictly less than $dist[u]$ at that point — but a smaller distance means v would have been popped from the priority queue before u , a direct

contradiction, since u was assumed to be popped first. This proof relies critically on non-negative weights: Week 11 Lecture 4's negative-weight counterexample breaks exactly this step, since a later-discovered path can be cheaper precisely because a negative edge can make $dist[v]$ not smaller than $dist[u]$ at the relevant moment, invalidating the contradiction.

Cost analysis, and why priority queue choice matters

Each node is popped from the priority queue at most once, guarded by the `visited` check, giving $O(V)$ total pops, each costing $O(\log V)$ using Week 8's heap-backed priority queue, for $O(V \log V)$ total. Each edge can trigger at most one `heappush`, when it produces a genuine distance improvement, giving $O(E)$ total pushes, each also $O(\log V)$, for $O(E \log V)$ total. Summing gives $O((V + E) \log V)$ overall. Using a plain unsorted array instead of a heap for the priority queue, scanning linearly for the minimum each time, would cost $O(V^2)$ instead — dramatically worse for the sparse graphs Week 11 Lecture 1 established as typical of real-world data, directly illustrating why the specific choice of supporting structure matters, not merely the high-level algorithm design.

MTech addition — Dijkstra's as a greedy algorithm

Dijkstra's algorithm is, structurally, a genuine greedy algorithm: at every step, it makes a locally greedy choice, finalizing whichever unvisited node currently has the smallest known distance, and never reconsiders that choice afterward. This fits Week 10's exact framework precisely — a greedy choice justified by its own specific correctness proof, not by loose resemblance to a previously correct greedy algorithm, exactly Week 10 Lecture 4's non-negotiable discipline. The non-negative-weight requirement is precisely the condition under which the greedy-choice property, defined in Week 10 Lecture 1, actually holds for this specific problem — removing that condition, as Week 11 Lecture 4's negative-weight example showed, removes the property entirely, and the greedy strategy becomes provably incorrect.

Real-world use of Dijkstra's algorithm

Dijkstra's algorithm is genuinely deployed at massive scale, not merely a classroom exercise. GPS navigation systems use it directly, with edge weights representing real travel time or distance — exactly this lecture's postal-delivery analogy, operating at global scale across road networks with millions of nodes. OSPF (Open Shortest Path First), a real, widely used internet routing protocol, uses Dijkstra's algorithm directly to compute shortest paths between network routers. Any “cheapest way to combine resources” problem with non-negative costs is a candidate application — this is a load-bearing, foundational algorithm underlying real infrastructure, not an isolated textbook construction.

Exercise

As an exercise, implement `dijkstra` and `reconstruct_path` exactly as presented in this lecture, verifying against the traced example: `dist = {0: 0, 1: 2, 2: 1}`, with the reconstructed path to node 1 being `[0, 2, 1]`. Separately, construct a 5-node weighted graph of your own design, compute all shortest distances from a chosen start node by hand first, and then verify your code's output matches your hand computation exactly.

Single-source vs. all-pairs: a distinction worth naming

It is worth explicitly naming a distinction: Dijkstra's algorithm, as covered in this lecture, is a single-source shortest-path algorithm, computing distances from one designated starting node to every other reachable node. A genuinely different question asks for the shortest distance between every pair of nodes simultaneously — the all-pairs shortest-path problem. A naive approach runs Dijkstra's algorithm once per starting node, $O(V)$ times, giving $O(V(V + E) \log V)$ total. Specialized all-pairs algorithms, such as Floyd-Warshall, beyond this course's scope, can outperform this naive repetition specifically for dense graphs — worth knowing this distinction and this further algorithm exist, even without covering it in full here.

A note on ties: multiple shortest paths

If two genuinely different paths to the same node happen to have exactly equal total weight, `dist` correctly records that shared minimum value regardless of which path is discovered first. `prev` records whichever path happened to be discovered and pushed to the priority queue first, producing a genuinely valid shortest path, but not necessarily the only one that achieves the minimum distance. This directly mirrors Week 9 Lecture 3's closing observation about longest common subsequence: the optimal *value* is always unique, but the specific optimal *path* or *sequence* achieving it may not be, and reconstruction returns one valid answer among possibly several.

Quiz — will this node's distance ever change?

Given that node u has just been popped from the priority queue and marked visited with `dist[u] = 7`, and later an edge from some other node is discovered that would offer u a distance of 5, determine before checking the answer whether `dist[u]` is updated to 5.

Quiz — answer

No, `dist[u]` does not update. The algorithm's explicit `if neighbor in visited: continue` check skips any already-visited node entirely, regardless of what distance a newly discovered edge might offer. This is correct precisely because the correctness proof established earlier in this lecture guarantees a visited node's distance is already truly optimal, assuming all edge weights are non-negative. If this guarantee did not hold — exactly the negative-weight case from Week 11 Lecture 4 — skipping already-visited nodes would silently produce a wrong answer, which is exactly why Dijkstra's algorithm is unsound on graphs containing negative weights.

A closing connection to Week 10's minimum spanning tree

It is worth closing with a direct comparison to Week 10 Lecture 3's minimum spanning tree material, which briefly previewed Prim's algorithm. Both Dijkstra's algorithm and Prim's algorithm use an essentially identical loop structure: a priority queue, repeatedly extracting the current minimum and updating neighboring distances. The genuine difference is what each tracks: Dijkstra's tracks the accumulated total distance from the single source node, while Prim's tracks only the weight of the single cheapest edge connecting the already-built tree to a new node, with no accumulation at all. Confusing the two is a common and real implementation bug, precisely because the code looks nearly identical while optimizing genuinely different quantities — a subtlety worth being explicitly aware of.

Verifying the correctness proof against the specific counterexample

It is worth being precise about exactly which claim this lecture resolved. Week 11 Lecture 4's failing graph used edge weights of 10, 1, and 1 — all non-negative, satisfying Dijkstra's correctness requirement fully. The proof's key step, that any shorter path passes through a node with smaller known distance, holds exactly here: node 2, with distance 1, is correctly popped before node 1, and examining its edge correctly reveals the better path to node 1. The earlier failure demonstrated in Week 11 was specifically BFS's failure, from treating a weighted graph as unweighted — Dijkstra's algorithm itself was never the algorithm that failed on this graph; it was never run there at all until this lecture.

Summary

Dijkstra's algorithm replaces BFS's plain queue with Week 8's priority queue, finalizing whichever unvisited node currently has the smallest known distance at every step. Its correctness rests critically on non-negative edge weights, without which the core "any shorter path passes through a smaller-distance node" argument breaks down entirely. Its cost is $O((V + E) \log V)$ using a heap-backed priority queue, dramatically better than the $O(V^2)$ a naive linear-scan priority queue would cost. This directly and completely resolves Week 11 Lecture 4's demonstrated BFS failure, providing both correct shortest distances and reconstructable actual paths. Next lecture introduces A* search, which uses

problem-specific heuristic knowledge to search even faster than plain Dijkstra's algorithm in many practical settings.